PROJET FINAL
Sujet : WaveGuard
"Système de Détection de Fraude Mobile Money en Temps Réel"

Objectif du projet:
→ Déployer et configurer un pipeline Big Data temps réel avec Docker
→ Mise en oeuvre de Apache Kafka : topics, partitions, producers/consumers
→ Implémenter Spark Structured Streaming avec watermark et fenêtres temporelles
→ Garantir la tolérance aux pannes via les checkpoints (exactly-once)
→ Mettre en place un monitoring avec Grafana
→ Pour aller plus loin/Optionnel : Sécuriser un pipeline Kafka avec SASL et ACL

Consignes (Important!):

1. Travail individuel ou par binôme.
2. Lisez tout le sujet, des annexes sont fournies en fin de document.
3. Les commandes doivent être exécutées dans l'environnement Docker fourni.
4. Un rapport de synthèse (README.md) est à remettre à la fin avec : captures d'écran clés + réponses aux questions ouvertes.
5. Les questions ouvertes (Q) évaluent votre compréhension conceptuelle — répondez avec précision.
6. Le code doit être commenté et fonctionnel.
7. Date limite de remise : vendredi 24 Avril à 08h00.

Mise en situation / Contexte:
Vous êtes ingénieurs Data chez TechFinSN, partenaire technologique d'un opérateur de Mobile Money sénégalais (inspiré de Wave / Orange Money / Free Money).
Votre mission : construire *WaveGuard*, un système de détection de fraude en temps réel capable de traiter des millions de transactions par jour.
Le système doit détecter deux types de fraudes :

- Fraude par vélocité : >5 transactions depuis un même compte en moins de 5 minutes
- Fraude par volume : montant total >500 000 FCFA depuis un compte sur 10 minutes

Les alertes doivent être générées en moins de 2 secondes. Le système doit résister aux pannes.

TAF:
Le travail attendu est structuré en quatre (04) parties principalement:

1. Partie 1 : Ingestion Kafka
   a. Création des 3 topics, code du Producer Python complet,
   b. Trois (03) questions ouvertes (Q1 à Q3)
2. Partie 2 : Spark Structured Streaming
   a. Règle 1 (code fourni), Règle 2 (à implémenter), sinks multiples (Kafka + Parquet),
   b. Questions sur outputMode et sliding window (Q4, Q5)
3. Partie 3 : Tolérance aux pannes
   a. Procédure de crash/reprise pas à pas, questions sur exactly-once (Q6, Q7)
4. Partie 4 ; Monitoring Grafana
   a. Export de métriques, dashboard à construire, alertes (Q8)
5. Optionnelle : Sécurité Kafka
   a. SASL/PLAIN + ACLs + test de refus d'accès (Q9)

PARTIE 0 — Mise en place de l'environnement
Avant tout, vérifiez que votre environnement Docker fonctionne correctement. Toute la stack est pré-configurée dans le fichier docker-compose.yml fourni.

0.1 Démarrage de la stack
Placez-vous dans le répertoire contenant le docker-compose.yml et exécutez :

```
# Lancer tous les services en arrière-plan
docker compose up -d

# Vérifier que tous les conteneurs sont UP
docker compose ps
```

Vous devez voir les services suivants avec le statut Up :

Service	Image	Port exposé	Rôle
zookeeper	confluentinc/cp-zookeeper	2181	Coordination Kafka
kafka	confluentinc/cp-kafka	9092	Message Broker
spark	bitnami/spark	8080	Traitement Streaming
grafana	grafana/grafana	3000	Monitoring / Dashboard
airflow	apache/airflow	8081	Orchestration

0.2 Vérification des accès

```
# Tester la connexion Kafka (depuis l'intérieur du conteneur)
docker exec -it <kafka_container_name> kafka-topics --bootstrap-server localhost:9092 --list

# Interface Spark Master
# Ouvrez un navigateur : http://localhost:8080

# Interface Grafana
# Ouvrez un navigateur : http://localhost:3000  (admin / admin par défaut)
```

0.3 Installation des dépendances Python (si non encore faite sur votre machine hôte)

```
pip install confluent-kafka pyspark==3.4.0 faker
```

NB.
Si Spark tourne dans le conteneur Docker, vous devrez soumettre vos jobs avec spark-submit depuis l'intérieur du conteneur, ou configurer le remote submission, ou plus simplement utiliser votre notebook. (L'ajouter au docker compose comme on le faisait aux TPs)

PARTIE 1 — Ingestion Kafka

Dans cette partie, vous allez configurer Apache Kafka pour ingérer les transactions Mobile Money et concevoir le Producer Python qui simule le trafic réel.

1.1 Création des topics

WaveGuard nécessite trois topics Kafka avec des configurations distinctes. Créez-les avec les commandes suivantes :

```
# Remplacez <kafka_container> par le nom réel du conteneur Kafka (voir étape 0.1)

# Topic principal : flux de transactions
docker exec <kafka_container> kafka-topics \
  --bootstrap-server localhost:9092 --create \
  --topic transactions \
  --partitions 3 \
  --replication-factor 1

# Topic des alertes de fraude
docker exec <kafka_container> kafka-topics \
  --bootstrap-server localhost:9092 --create \
  --topic fraud-alerts \
  --partitions 1 \
  --replication-factor 1

# Topic d'audit
docker exec <kafka_container> kafka-topics \
  --bootstrap-server localhost:9092 --create \
  --topic audit-log \
  --partitions 2 \
  --replication-factor 1

# Vérification
docker exec <kafka_container> kafka-topics \
  --bootstrap-server localhost:9092 --describe --topic transactions
```

Q1. Justifiez le choix de 3 partitions pour le topic transactions et de 1 seule partition pour fraud-alerts. Quel est l'impact sur l'ordre de traitement ?

1.2 Le Producer Python — Simulateur de transactions

Implémentez le fichier producer.py ci-dessous. Ce code vous est fourni comme point de départ ; vous devrez l'enrichir.

```
# producer.py — Simulateur de transactions Mobile Money WaveGuard
from confluent_kafka import Producer
from faker import Faker
```

```
import json, time, random, uuid
from datetime import datetime, timezone

fake = Faker('fr_FR')
BROKER = 'localhost:9092'
TOPIC  = 'transactions'

# Liste de comptes (certains seront des fraudeurs)
ACCOUNTS = [f'SN_{i:04d}' for i in range(1, 51)]
FRAUD_ACCOUNTS = ['SN_0042', 'SN_0007', 'SN_0013']  # comptes suspects

conf = {'bootstrap.servers': BROKER}
producer = Producer(conf)

def delivery_report(err, msg):
    if err:
        print(f'[ERREUR] Livraison échouée : {err}')
    else:
        print(f'[OK] Topic={msg.topic()} | Partition={msg.partition()} | Offset={msg.offset()}')

def generate_transaction(fraud=False):
    sender = random.choice(FRAUD_ACCOUNTS if fraud else ACCOUNTS)
    amount = random.randint(800_000, 2_000_000) if fraud else random.randint(500,
150_000)
    return {
        'transaction_id': str(uuid.uuid4()),
        'timestamp':      datetime.now(timezone.utc).isoformat(),
        'sender_id':      sender,
        'receiver_id':    random.choice(ACCOUNTS),
        'amount_fcfa':    amount,
        'transaction_type': random.choice(['P2P', 'PAIEMENT_MARCHAND', 'RETRAIT']),
        'location':       random.choice(['Dakar', 'Thiès', 'Saint-Louis', 'Ziguinchor', 'Kaolack']),
    }

print('=== WaveGuard Producer démarré ===  Ctrl+C pour arrêter')
try:
    while True:
        # Injection de fraude : 10% de chance d'envoyer une transaction suspecte
        is_fraud = random.random() < 0.10
        tx = generate_transaction(fraud=is_fraud)
```

```
        # TODO Partie 1.3 : implémenter le burst frauduleux

        producer.produce(
            TOPIC,
            key=tx['sender_id'],          # clé = partition par compte
            value=json.dumps(tx).encode(),
            callback=delivery_report
        )
        producer.poll(0)
        time.sleep(random.uniform(0.05, 0.3))   # ~5 à 20 tx/sec

except KeyboardInterrupt:
    print('Arrêt du producer...')
    producer.flush()
```

1.3 Travail demandé — Producer enrichi

Complétez et enrichissez le producer.py pour :

1. Modifier le code pour que les comptes FRAUD_ACCOUNTS envoient des *bursts* : 8 transactions en rafale espacées de seulement 50ms (simulant une attaque par vélocité).
2. Ajouter un champ is_flagged (booléen) dans le message JSON, positionné à True pour les comptes fraudeurs. Cela servira de vérité-terrain pour valider votre détection.
3. Implémenter une logique de partitionnement explicite : toutes les transactions d'un même sender_id doivent aller dans la même partition. Vérifiez via les logs de delivery_report.
4. Faire tourner votre producer. Vérifier la réception avec le consumer console de Kafka :

```
docker exec <kafka_container> kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic transactions \
  --from-beginning --max-messages 10
```

Q2. Kafka garantit-il l'ordre des messages à l'échelle du topic entier ? Quelle configuration du Producer permettrait de s'assurer que toutes les transactions d'un compte arrivent dans la bonne partition ?

Q3. Comparez Kafka et RabbitMQ pour le cas d'usage WaveGuard. Lequel choisiriez-vous et pourquoi ?

PARTIE 2 — Spark Structured Streaming

Dans cette partie, vous allez construire le cœur du système WaveGuard : le moteur de détection de fraude avec Spark Structured Streaming.

2.1 Lecture du flux Kafka avec Spark

Créez le fichier waveguard_detector.py. Commencez par établir la connexion Kafka → Spark :

```
# waveguard_detector.py
```

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    from_json, col, window, count, sum as spark_sum,
    current_timestamp, lit, to_json, struct
)
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType,
    TimestampType, BooleanType
)

# ─── Configuration Spark ──────────────────────────────────────────────────────
spark = SparkSession.builder \
    .appName('WaveGuard_FraudDetector') \
    .config('spark.jars.packages',
            'org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.0') \
    .config('spark.sql.streaming.checkpointLocation', '/tmp/waveguard_checkpoint') \
    .getOrCreate()

spark.sparkContext.setLogLevel('WARN')

# ─── Schéma JSON des transactions ─────────────────────────────────────────────
schema = StructType([
    StructField('transaction_id',   StringType(),    True),
    StructField('timestamp',        TimestampType(), True),
    StructField('sender_id',        StringType(),    True),
    StructField('receiver_id',      StringType(),    True),
    StructField('amount_fcfa',      DoubleType(),    True),
    StructField('transaction_type', StringType(),    True),
    StructField('location',         StringType(),    True),
    StructField('is_flagged',       BooleanType(),   True),
])

# ─── Lecture du flux Kafka ────────────────────────────────────────────────────
raw_stream = spark.readStream \
    .format('kafka') \
    .option('kafka.bootstrap.servers', 'localhost:9092') \
    .option('subscribe', 'transactions') \
    .option('startingOffsets', 'latest') \
    .load()
```

```
# ——— Parsing et extraction ————————————————————————————————————
parsed_df = raw_stream \
    .selectExpr('CAST(value AS STRING) as json_str', 'timestamp as kafka_ts') \
    .select(from_json(col('json_str'), schema).alias('data'), col('kafka_ts')) \
    .select('data.*', 'kafka_ts')

# ——— Watermark : tolérance aux messages en retard ————————————————————
df_with_watermark = parsed_df.withWatermark('timestamp', '2 minutes')
```

2.2 Règle 1 — Fraude par vélocité

Implémentez la détection basée sur la fréquence de transactions dans une fenêtre glissante :

```
# ——— RÈGLE 1 : Fraude par vélocité ————————————————————————————————
# > 5 transactions depuis un même compte en 5 minutes (slide: 1 min)
velocity_fraud = df_with_watermark \
    .groupBy(
        window(col('timestamp'), '5 minutes', '1 minute'),  # fenêtre glissante
        col('sender_id')
    ) \
    .agg(count('*').alias('tx_count')) \
    .filter(col('tx_count') > 5) \
    .select(
        col('sender_id'),
        col('window.start').alias('window_start'),
        col('window.end').alias('window_end'),
        col('tx_count'),
        lit('VELOCITY_FRAUD').alias('fraud_type'),
        current_timestamp().alias('detected_at')
    )
```

2.3 Règle 2 — Fraude par volume (à implémenter)

Implémentez vous-même la règle de détection par volume en vous inspirant de la Règle 1 :

Spécification -- règle 2 : Volume:

- Fenêtre : 10 minutes, slide de 2 minutes
- Seuil : montant total > 500 000 FCFA depuis un même sender_id
- Champ fraud_type à valoriser : 'VOLUME_FRAUD'
- Champs à inclure dans le résultat : sender_id, window_start, window_end, total_amount (somme des montants), fraud_type, detected_at

Q4. Expliquez la différence entre une *Tumbling Window* et une *Sliding Window*. Pourquoi choisit-on une sliding window pour la détection de fraude ? Donnez un exemple concret avec les paramètres du TP.

2.4 Écriture des alertes — Sink multiples

Configurez deux sinks pour les alertes détectées :

```
# ——— SINK 1 : Écriture vers le topic Kafka 'fraud-alerts' ————
def write_to_kafka(df, fraud_type_label):
    return df \
        .select(to_json(struct('*')).alias('value')) \
        .writeStream \
        .format('kafka') \
        .option('kafka.bootstrap.servers', 'localhost:9092') \
        .option('topic', 'fraud-alerts') \
        .option('checkpointLocation', f'/tmp/checkpoint_{fraud_type_label}') \
        .outputMode('update') \
        .start()

# ——— SINK 2 : Écriture dans le Data Lake local (Parquet) ————
def write_to_datalake(df, fraud_type_label):
    return df \
        .writeStream \
        .format('parquet') \
        .option('path', f'/tmp/waveguard_lake/{fraud_type_label}') \
        .option('checkpointLocation', f'/tmp/checkpoint_lake_{fraud_type_label}') \
        .outputMode('append') \
        .trigger(processingTime='30 seconds') \
        .start()

# ——— Lancement des queries ————————————————————————————————

q1_kafka = write_to_kafka(velocity_fraud, 'velocity')
# TODO : ajouter volume_fraud

# Attendre la fin (Ctrl+C pour arrêter)
spark.streams.awaitAnyTermination()
```

Q5. Pour le sink Parquet (Data Lake), on utilise outputMode('append') alors que pour le sink Kafka on utilise outputMode('update'). Expliquez pourquoi. Pourquoi le mode 'complete' serait-il problématique ici ?

PARTIE 3 — Tolérance aux pannes & Exactly-Once

Un système de détection de fraude financière ne peut pas perdre de données ni traiter une transaction deux fois. Cette partie valide votre compréhension des mécanismes de résilience.

3.1 Mise en place du checkpoint

Le checkpoint Spark sauvegarde périodiquement l'état du streaming. Votre code intègre déjà la configuration de base. Vérifiez que le répertoire de checkpoint se crée bien :

```
# Après avoir lancé waveguard_detector.py, vérifiez :
ls -la /tmp/waveguard_checkpoint/

# Vous devez voir des fichiers de metadata, offsets, et commits
# Structure type :
#   /tmp/waveguard_checkpoint/
#   ├── commits/        ← batches traités avec succès
#   ├── metadata        ← infos de la query
#   ├── offsets/        ← offsets Kafka lus
#   └── sources/        ← état des sources
```

3.2 Simulation de crash et reprise

Suivez la procédure suivante pour simuler un crash et valider la reprise :

i. Lancez votre producer (python producer.py) et votre detector (spark-submit waveguard_detector.py) simultanément.

ii. Laissez tourner 2-3 minutes. Notez l'offset courant via :

```
docker exec <kafka_container> kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group spark-waveguard \
  --describe
```

iii. Simulez un crash : tuez brutalement le job Spark.

```
# Trouver le PID du job Spark
ps aux | grep waveguard
kill -9 <PID>
```

iv. Attendez 30 secondes (le producer continue d'envoyer des transactions).

v. Relancez le detector (sans modifier le checkpoint). Observez les logs : Spark doit reprendre exactement là où il s'était arrêté.

vi. Vérifiez que toutes les transactions émises pendant la panne sont bien traitées.

Q6. Expliquez le mécanisme de checkpoint Spark. Comment Spark sait-il exactement quels offsets Kafka reprendre après un crash ? Que se passerait-il si vous supprimiez le répertoire de checkpoint avant le redémarrage ?

Q7. Décrivez les conditions précises à réunir dans votre pipeline pour atteindre la garantie *exactly-once end-to-end* (du Producer Kafka au sink Parquet). Quelles sont les limitations potentielles ?

PARTIE 4 — Monitoring Grafana

Un pipeline de production sans monitoring est un pipeline aveugle. Cette partie vous familiarise avec Grafana pour observer votre système WaveGuard en temps réel.

4.1 Connexion à Grafana

i. Ouvrez Grafana dans votre navigateur : http://localhost:3000
ii. Connectez-vous avec les identifiants par défaut : admin / admin (changez le mot de passe si demandé).
iii. Allez dans Configuration > Data Sources > Add data source.

4.2 Source de données — Fichiers Parquet

Pour ce travail, vous allez utiliser les fichiers Parquet générés par Spark comme source de données. La solution la plus simple consiste à exposer les métriques via un fichier JSON intermédiaire :

```
# metrics_exporter.py — À lancer en parallèle de votre pipeline
import time, json, os
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('MetricsExporter').getOrCreate()
METRICS_FILE = '/tmp/waveguard_metrics.json'

while True:
    try:
        # Lire les alertes générées
        df_velocity = spark.read.parquet('/tmp/waveguard_lake/velocity')
        df_volume   = spark.read.parquet('/tmp/waveguard_lake/volume')

        metrics = {
            'timestamp':       time.time(),
            'velocity_alerts': df_velocity.count(),
            'volume_alerts':   df_volume.count(),
            'top_fraudster':   df_velocity.groupBy('sender_id').count()
                               .orderBy('count', ascending=False)
                               .first()['sender_id'] if df_velocity.count() > 0 else 'N/A'
        }
        with open(METRICS_FILE, 'w') as f:
            json.dump(metrics, f)
        print(f'Métriques exportées : {metrics}')
    except Exception as e:
        print(f'Erreur : {e}')
    time.sleep(15)  # Actualisation toutes les 15 secondes
```

4.3 Création du dashboard WaveGuard

Créez un dashboard Grafana avec les panels suivants :

Panel	Type de visualisation	Métrique affichée	Seuil d'alerte
Alertes Vélocité	Stat (compteur)	Nb total alertes vélocité	> 10 alertes
Alertes Volume	Stat (compteur)	Nb total alertes volume	> 5 alertes
Top Fraudeur	Text / Table	Compte le plus actif	—
Transactions/min	Time Series	Volume temps réel	—

4.4 Configuration d'une alerte Grafana

i. Dans Grafana, accédez à Alerting > Alert Rules > New Alert Rule.

ii. Configurez une alerte qui se déclenche si le nombre d'alertes de fraude dépasse 10 en moins de 5 minutes.

iii. Configurez une notification (channel : Email ou Slack selon disponibilité).

Q8. Dans un environnement de production, quelle(s) source(s) de métriques Kafka et Spark recommanderiez-vous d'utiliser avec Grafana ? Décrivez l'architecture de monitoring complète.

Op. Sécurisation du Pipeline Kafka:

Cette partie optionnelle traite de la sécurisation du pipeline, un aspect critique pour un système financier. *Elle peut vous rapporter des points supplémentaires*.

Op.1 Activation de SASL/PLAIN

Modifiez le docker-compose.yml pour activer l'authentification SASL :

```
# docker-compose.yml — Section kafka (version sécurisée)
kafka:
  image: confluentinc/cp-kafka:latest
  depends_on:
    - zookeeper
  environment:
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: SASL_PLAINTEXT://kafka:9092
    KAFKA_SECURITY_INTER_BROKER_PROTOCOL: SASL_PLAINTEXT
    KAFKA_SASL_ENABLED_MECHANISMS: PLAIN
    KAFKA_SASL_MECHANISM_INTER_BROKER_PROTOCOL: PLAIN
    KAFKA_OPTS: >-
      -Djava.security.auth.login.config=/etc/kafka/kafka_server_jaas.conf
  volumes:
    - ./kafka_server_jaas.conf:/etc/kafka/kafka_server_jaas.conf
  ports:
    - '9092:9092'
# kafka_server_jaas.conf (à créer dans le même répertoire)
KafkaServer {
  org.apache.kafka.common.security.plain.PlainLoginModule required
```

```
  username='admin'
  password='admin-secret'
  user_admin='admin-secret'
  user_spark='spark-secret'
  user_producer='producer-secret';
};
```

Op.2 Définition des ACLs

```
# Autoriser le producer à écrire sur 'transactions'
docker exec <kafka_container> kafka-acls \
  --bootstrap-server localhost:9092 \
  --command-config /tmp/admin.properties \
  --add \
  --allow-principal User:producer \
  --operation Write \
  --topic transactions

# Autoriser Spark à lire 'transactions' dans un consumer group
docker exec <kafka_container> kafka-acls \
  --bootstrap-server localhost:9092 \
  --command-config /tmp/admin.properties \
  --add \
  --allow-principal User:spark \
  --operation Read \
  --topic transactions \
  --group spark-waveguard

# Interdire au producer de lire (par défaut refus, mais explicitons)
docker exec <kafka_container> kafka-acls \
  --bootstrap-server localhost:9092 \
  --command-config /tmp/admin.properties \
  --add \
  --deny-principal User:producer \
  --operation Read \
  --topic transactions
```

Op.3 Test de refus d'accès

Tentez de lire le topic transactions avec le compte producer (qui ne devrait pas y avoir accès en lecture) :

```
# Créer producer.properties avec les credentials producer
# security.protocol=SASL_PLAINTEXT
# sasl.mechanism=PLAIN
# sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
#   required username='producer' password='producer-secret';

# Tentative de lecture avec le compte producer → doit être REFUSÉE
docker exec <kafka_container> kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --consumer.config /tmp/producer.properties \
  --topic transactions

# Résultat attendu :
# [ERROR] Not authorized to access topics: [transactions]
```

Q9 [optionnelle]. Comparez SASL/PLAIN, SASL/SCRAM et SASL/GSSAPI (Kerberos). Dans quel contexte choisiriez-vous chacun pour un déploiement en production chez un opérateur Mobile Money ?

Livrable attendu (à remettre)

Un dépôt Git (ou archive ZIP) contenant :

- producer.py — le simulateur de transactions enrichi
- waveguard_detector.py — le moteur de détection Spark
- docker-compose.yml — éventuellement modifié pour le bonus sécurité
- README.md — réponses aux questions ouvertes (Q1 à Q9) + captures d'écran Grafana
- screenshots/ — captures : consumer console, logs Spark, dashboard Grafana, refus ACL

Date limite de remise : vendredi 24 Avril à 08h00.

Grille d'évaluation (Indicative)

#	Critère d'évaluation	Points	Indicateurs de réussite
P1	Topics Kafka créés et configurés correctement	/1	3 topics avec bonnes partitions + capture describe
P1	Producer Python fonctionnel avec burst frauduleux	/2	Transactions visibles dans consumer console, bursts injectés
P2	Règle 1 — Fraude par vélocité implémentée	/3	Alertes générées sur FRAUD_ACCOUNTS, topic fraud-alerts peuplé
P2	Règle 2 — Fraude par volume implémentée	/3	Seuil 500k FCFA détecté, sink Parquet fonctionnel

P3	Checkpoint et reprise après crash validés	/3	Offset repris correctement, aucune transaction perdue
P4	Dashboard Grafana avec alertes configurées	/2	4 panels présents + 1 alerte configurée
QO	Questions ouvertes (Q1 à Q8) — qualité des réponses	/4	Précision technique, justifications, maîtrise des concepts
Bonus	SASL, ACLs, refus d'accès démontré	/2	Capture du refus d'accès + réponse Q9

ANNEXE — Aide-mémoire & Commandes utiles

Commandes Kafka essentielles

```
# Lister les topics
docker exec <kafka> kafka-topics --bootstrap-server localhost:9092 --list
# Décrire un topic
docker exec <kafka> kafka-topics --bootstrap-server localhost:9092 --describe --topic <nom>
# Consumer console (depuis le début)
docker exec <kafka> kafka-console-consumer --bootstrap-server localhost:9092 --topic transactions --from-beginning
# Vérifier le consumer lag
docker exec <kafka> kafka-consumer-groups --bootstrap-server localhost:9092 --group spark-waveguard --describe
```

Commandes Docker utiles

```
docker compose ps                    # État des services
docker compose logs kafka            # Logs Kafka
docker compose logs spark            # Logs Spark
docker compose restart kafka         # Redémarrer un service
docker exec -it <container> bash     # Shell interactif
```

Soumission d'un job Spark

```
# Depuis l'hôte (si Spark configuré localement)
spark-submit \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.0 \
  waveguard_detector.py

# Depuis le conteneur Spark
docker exec -it <spark_container> spark-submit \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.0 \
```

```
/app/waveguard_detector.py
```

Schéma JSON — Format des transactions

```
{
  "transaction_id":   "uuid-v4",
  "timestamp":        "2025-11-15T14:32:00.000Z",
  "sender_id":        "SN_0042",
  "receiver_id":      "SN_0017",
  "amount_fcfa":      75000,
  "transaction_type": "P2P",
  "location":         "Dakar",
  "is_flagged":       false
}
```

Bons courage à tous !

Ce TP a été conçu pour vous challenger et vous préparer (big data et architecture) aux conditions réelles de l'industrie.

École Polytechnique de Thiès — GIT/DIC/Big Data — 2025/2026